

ROBOTICS

MEEC

Authors:

Beatriz Coelho (107113)

Daniel Burke (107107)

Diogo Sousa (107004)

Miguel Fialho (106193)

beatriz.h.coelho@tecnico.ulisboa.pt

daniel.burke@tecnico.ulisboa.pt

diogomppsousa@tecnico.ulisboa.pt

miguel.fialho@tecnico.ulisboa.pt

Group 30

2025/2026 – 1st Semester, P2

Introduction

Lane keeping assistance is a driver-support system that helps maintain a vehicle inside a road lane by correcting deviations in its trajectory. The purpose of this laboratory assignment was to design and simulate a Lane Tracing Assist (LTA) system for a car, applying concepts from robotics. In our implementation, we started from an existing Lane Keeping Assist example from MATLAB and used it as a base to build the architecture required by the project. The original steering model was replaced by a simplified kinematic bicycle model, which describes the motion of the vehicle using only the velocity and the steering angle.

The controller was replaced by a PD controller, which adjusts the steering based on the lateral deviation and heading angle. The activation of the LTA system was also developed to ensure smooth transitioning from the driver's inputs. A blinker system was integrated, to allow override of the LTA for lane switching. The perception part of the system was also extended, by integrating an additional lane detection method using a simulated onboard camera, which enabled the estimation of both lane boundaries from image data.

Our System can be illustrated by the following block diagram:

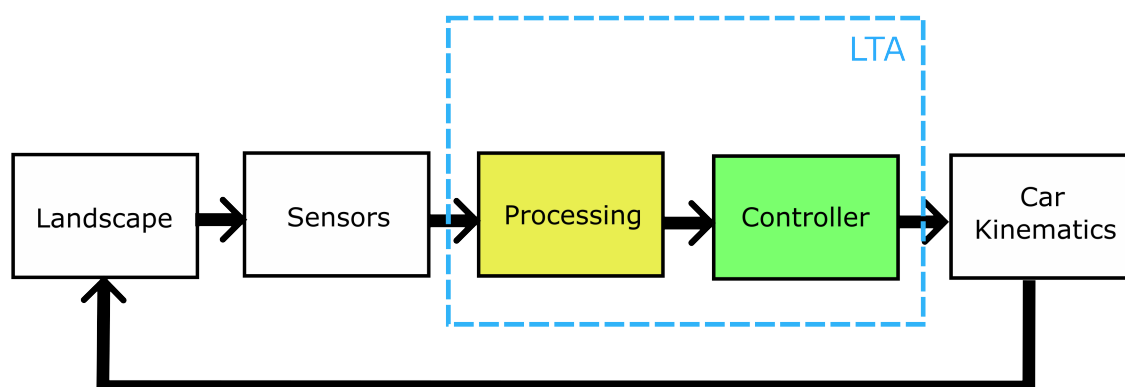


Figure 1: Block Diagram representation of our system.

Landscape

The landscape is made up with all the visual elements integrated in the simulator, and how these accommodate the actor, which in this case is the vehicle or car model. These must be well traced, so that the sensors can work accordingly.

Sensors

The Sensors are used to help with identifying the lane boundaries and the center of the lane. We use a simulated camera that is located in the center of the roof of the car. With this, the LTA system is able to make decisions about the current state of the vehicle.

Processing

With the inputs received from the camera, the processing unit can compute the current lateral deviation to the center of the lane that it sits in. It can also determine the yaw angle, which is the relation between the tangent of the lane boundary and the lane center.

Controller

If the lateral offset to the right or left lane is equal to a specific threshold, the controller will apply an assistance on the steering of the vehicle. The assisted steering correction angle is determined with basis on the lateral deviation and the yaw error.

Controls and Information display

The controls are the following:

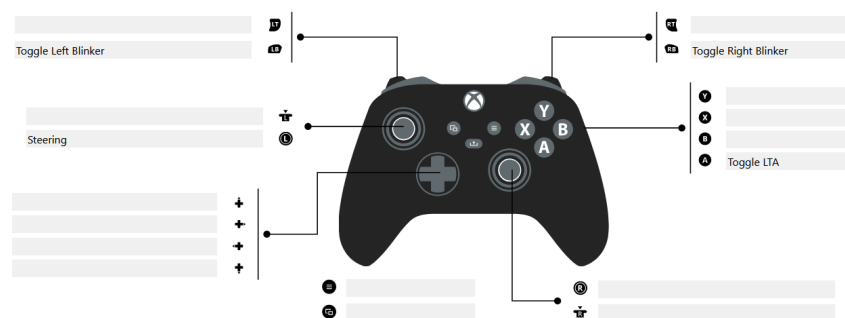


Figure 2: Gamepad Controls

The live information is displayed on the simulink window. The blinkers are also visible on the 3D car model. It's also possible to change the safe lateral distance, but it is already set to the value we considered best.

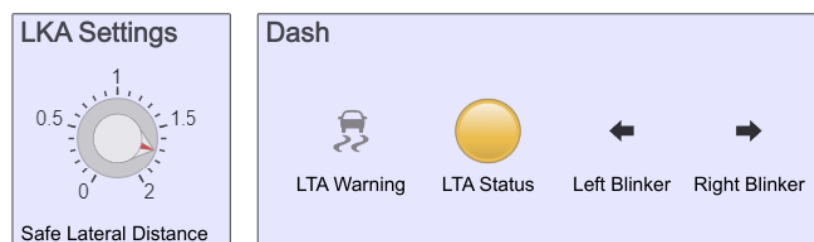


Figure 3: Dashboard

Car Kinematics

The Car Kinematics block represents how the vehicle moves in the simulated environment. It updates the position and orientation of the car and sends this information back into the system. This feedback allows the sensors and the LTA logic to understand the vehicle's current state on the road and adjust future steering decisions. In simple terms, this block closes the loop by translating the steering decisions into movement in the landscape.

Car Kinematic Model

To replace the dynamic vehicle model provided in the reference example, we implemented a simplified kinematic bicycle model. This model is directly based on the formulations presented in the course slides and focuses exclusively on the geometric motion of the vehicle, without considering dynamic effects such as mass, inertia, or tyre forces.

The vehicle is modelled as a rigid body moving on a planar surface and is described by its position (x, y) and its yaw angle ψ . The inputs to the model are the longitudinal velocity v and the steering angle δ .

Under the kinematic bicycle assumption, the yaw rate is given by:

$$\dot{\psi} = \frac{v}{L} \tan(\delta) \quad (1)$$

where L is the vehicle wheelbase. The vehicle position evolves according to:

$$\dot{x} = v \cos(\psi) \quad (2)$$

$$\dot{y} = v \sin(\psi) \quad (3)$$

The yaw angle is obtained by integrating the yaw rate over time:

$$\psi(t) = \int \dot{\psi}(t) dt \quad (4)$$

In the Simulink implementation, the steering angle is first passed through a low-pass filter to avoid abrupt changes that could lead to unrealistic motion. The yaw rate is then computed using the kinematic relation, and numerical integrators are used to obtain the yaw angle and the vehicle position at each simulation step.

Figure 4 illustrates the final Simulink implementation of the Car Kinematics block used in our system.

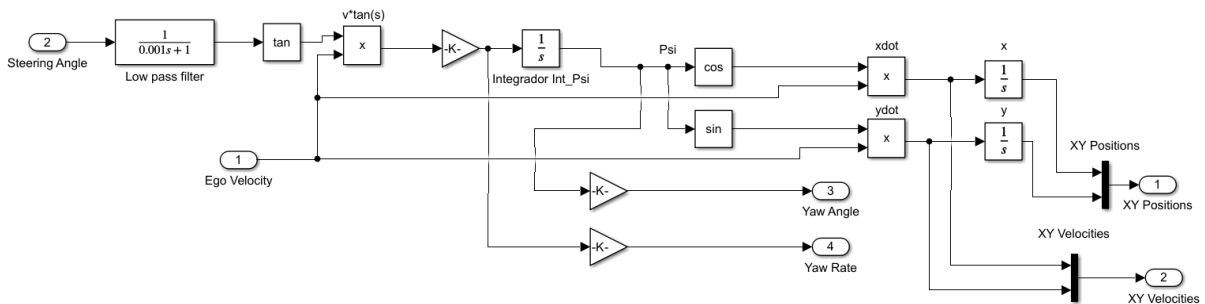


Figure 4: Car Model in Simulink

Kinematic vs Dynamic Model

To analyse the impact of replacing the original dynamic vehicle model with a simplified kinematic bicycle model, both models were simulated under identical conditions. During these tests, the Lane Tracing Assist system was disabled, so that the vehicle behavior depends exclusively on the car model itself and not on any control action.

The steering angle input was defined as a square wave with an amplitude of 0.1 rad and a frequency of 1 Hz. This input was selected to generate repeated steering changes and clearly expose the response characteristics of each model. The longitudinal velocity was kept constant during each simulation.

The comparison was performed using the Simulink Data Inspector, focusing on the vehicle trajectory in the XY plane, as well as on the yaw angle and yaw rate.

The dynamic model includes mass and rotational inertia, which limit how quickly the vehicle can change its orientation and lateral motion. In contrast, the kinematic model follows the geometric relations directly and reacts instantaneously to steering inputs.

In all figures, the dynamic model is shown in blue, while the kinematic model is shown in orange.

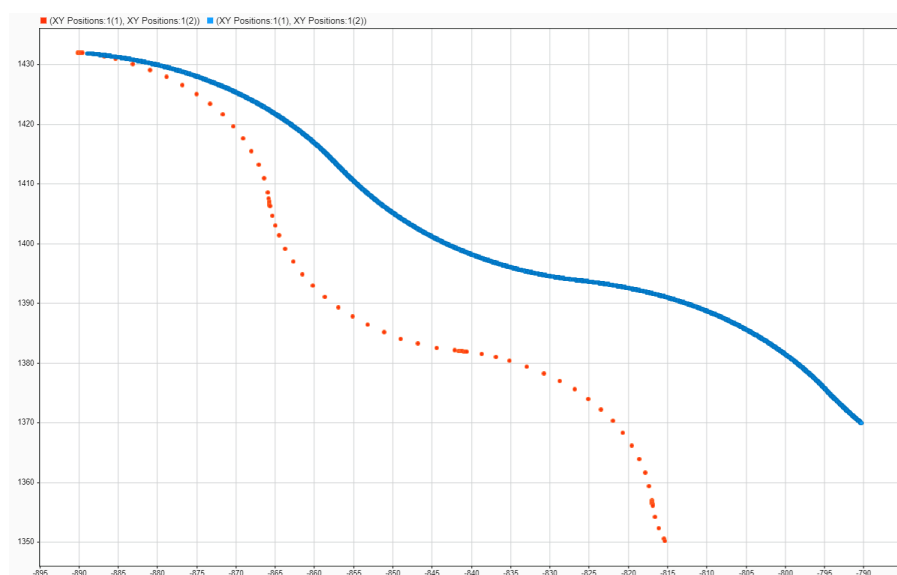


Figure 5: XY trajectory for both models at 12 m/s

At a speed of 12 m/s, both vehicle models respond to the square wave steering input by producing oscillatory motion in the XY plane. The main difference between the trajectories is related to how smoothly the vehicle changes direction.

The dynamic model (blue) produces a smoother trajectory because vehicle inertia limits how quickly lateral motion and orientation can change. The kinematic model (orange) reacts directly to the steering input, resulting in sharper curvature changes. At this speed, the effect of inertia is moderate, and therefore the overall motion of both models remains similar.

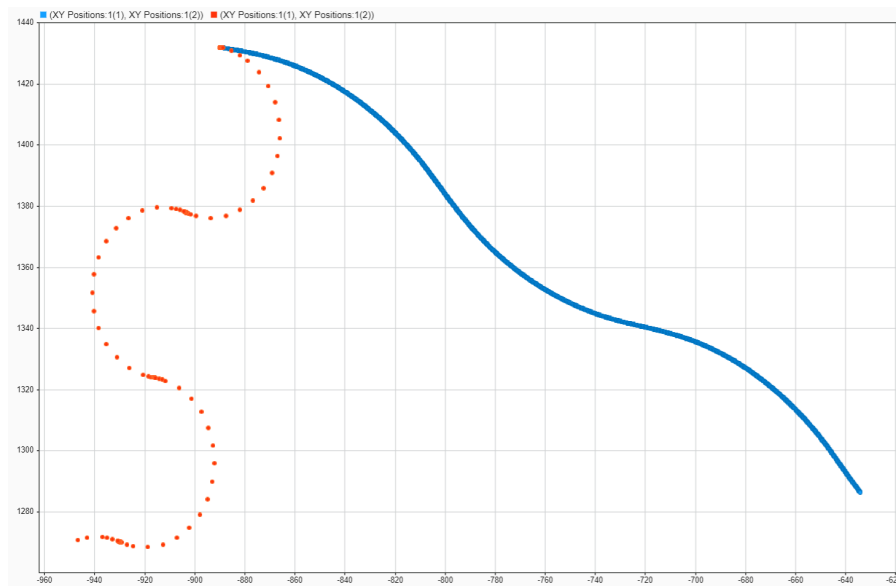


Figure 6: XY trajectory for both models at 30 m/s

At 30 m/s, the effect of inertia becomes much more significant. The dynamic model shows wider and smoother oscillations, as inertia strongly limits rapid changes in direction at higher speeds.

The kinematic model does not include inertial effects and therefore continues to follow the steering input directly, producing tighter and more abrupt motion. This clearly shows that the differences between the two models increase with vehicle speed due to inertial effects.

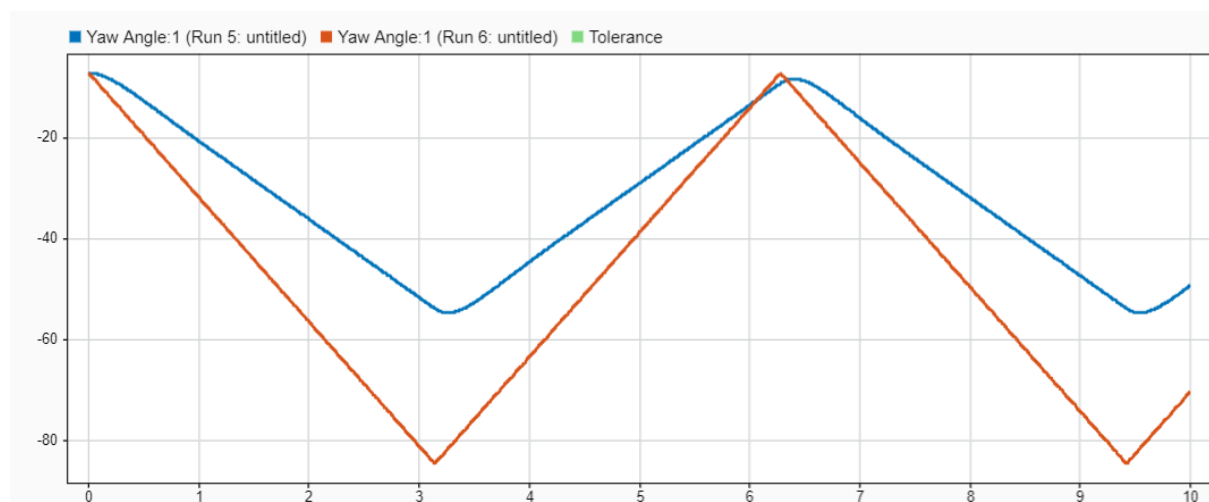


Figure 7: Yaw angle comparison at 12 m/s

The yaw angle comparison at 12 m/s highlights the different dynamic behaviour of the two vehicle models. The kinematic model follows the square wave steering input more directly, resulting in faster and more abrupt changes in yaw angle.

The dynamic model exhibits a smoother yaw angle evolution with a noticeable delay. This delay is caused by rotational inertia, which limits how quickly the vehicle can change its orientation. As a result, sudden steering changes are filtered by the vehicle dynamics, leading to a more gradual response.

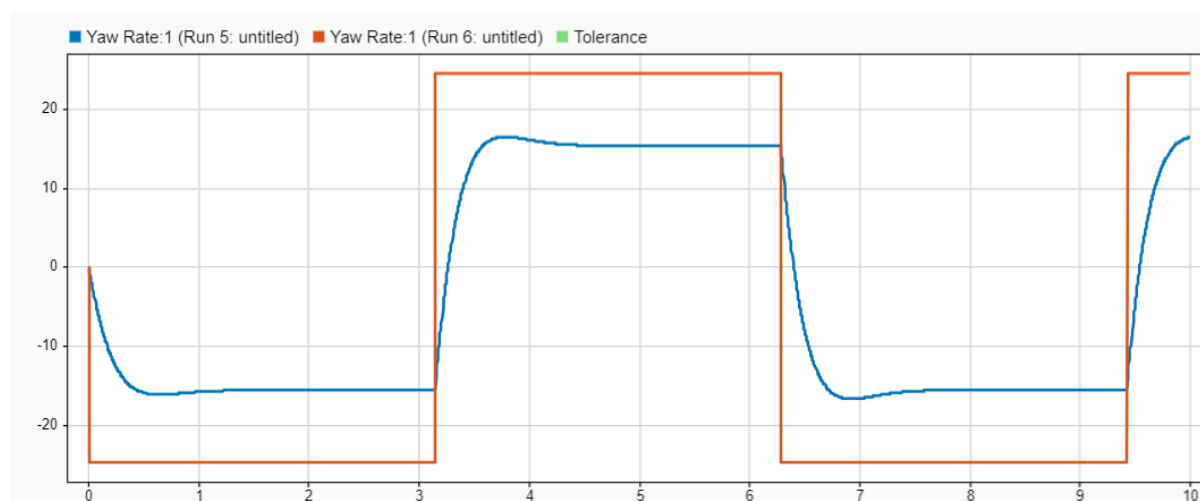


Figure 8: Yaw rate comparison at 12 m/s

The yaw rate response further clarifies this difference. Since the yaw rate is the time derivative of the yaw angle, it directly reflects how fast the vehicle rotates. The kinematic model produces almost instantaneous changes in yaw rate when the steering input switches, resulting in a more abrupt and piecewise response.

In contrast, the dynamic model shows smoother and bounded yaw rate variations. The presence of rotational inertia prevents sudden changes in angular velocity, causing the yaw rate to increase and decrease progressively.

Overall, the results show that the kinematic model reacts faster to steering inputs due to the absence of inertial effects, while the dynamic model presents a slower and smoother response consistent with real vehicle behavior.

Sensors and Processing

On Lane Tracing Assist systems implemented in vehicles like cars, lane detection is an essential part to have a robust system. For the car to be able to steer back into its lane it needs to know where the lane markings are in relation to itself. That's exactly what lane detection does, it "translates" the physical road markings into mathematical variables that are then used by the controller as input. To make this translation there are 2 steps: sensor acquisition and data processing.

- **Sensor acquisition:** There are several types of sensor that can be used in this application such as cameras, radar and LIDAR (or light detection and ranging). Most modern system on today's cars use a combination of cameras and radar where the camera is used more on the lane detection side and radar helps estimate the distance between the car and any vehicle in front for features like cruise control. LIDAR is very useful since it creates a point cloud of surroundings of the car and allows it to detect objects and obstacles with high precision, but can be very expensive and require a lot of processing power.
- **Data processing:** This is where the actual translation happens. The data from the sensors is cleaned, filtered and then image processing methods or learning-based methods are used to give a prediction of where the road markings are.

Original Implementation

In the original *Matlab* example, the lane detection system was composed of 2 blocks: “Scenario Reader” and “Vision Detection Generator”. Instead of doing sensor acquisition and data processing, this system simply takes the coordinates of the vehicle and road (ground truth), uses the position of the lanes to calculate the parameters need for the LTA controller. This system also adds noise to “make the estimations more realistic”.

Custom Implementation

For our implementation we decided to use just a camera since in our landscape and road scenario there are no obstacles, the only vehicle present is ours and Radar and LIDAR wouldn't help much in lane detection. For the camera we decided to place it on the hood of the car.



Figure 9: POV of the camera mounted on the hood of the car.

The objective is to be able to extract 2 parameters out of each frame that the camera provides: **lateral deviation** (from the centerline) and **heading angle** (the difference between the direction of centerline and the vehicle). To do this we used an algorithm that does the following steps:

- **Image transformation:** It takes the frame directly from the camera and defines a region of interest, that is the part of the frame that we are actually interested in analyzing. In other words, ignoring the sky and the hood of the car in the image. Then, it warps the image onto a “top-down view”. This is done by a technique called *Inverse Perspective Mapping*.
- **Finding the lane markings:** From the “top-down view” we create a binary mask by scanning the image and looking for color contrast between the road (dark and grey) and the lanes markings (bright white and yellow). In the mask, the road gets assigned a “0” and the lane markings a “1”.

- **Coordinates Extraction:** Once we create the mask, we need to convert the pixels onto real world dimensions (meters) and in the reference frame of the camera. This is done by using the know camera intrinsics parameters (the K matrix) and the pixel coordinates of the mask.
- **Polynomial Fitting:** With the mask points converted to meters, we apply the RANSAC (*RANdom SAmple Consensus*) algorithm to try and fit a second degree polynomial ($y = Ax^2 + Bx + C$) to the points that make up the lane markings (one polynomial for the left lane marking and one for the right one). RANSAC first picks the minimum number of points needed to fit the polynomial model (3 in this case) at random, estimates a model with these points, checks how many points fit this curve and repeats this cycle until it finds the best fit for all points.
- **Checking:** This step assures that the estimations make sense relative to the estimations from the previous frame. It checks for the width between the polynomials and sudden changes that could be caused by rapid movements of the car. If these are below a certain threshold, the estimations are considered “valid” and “invalid” otherwise. For the “invalid” estimations (or when no line was found), we apply a *Kalman filter* that predicts where the lane would have been based on the position and velocity of the car.
- **Parameter calculation:** It takes the 2 polynomials (y_{left} and y_{right}) and from coefficients extracts the parameters that will be used by the controller:
 - Heading angle = B
 - Lateral offset = C

Validation

The performance of each implementation can be seen by just plotting their lane estimation against the real world lanes:

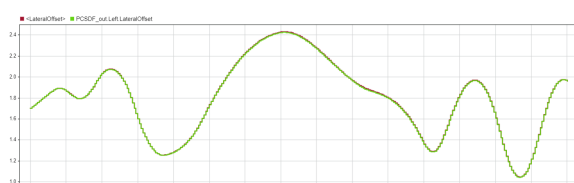


Figure 10: Left Lateral Offset: ground truth (purple) and original estimation (green).

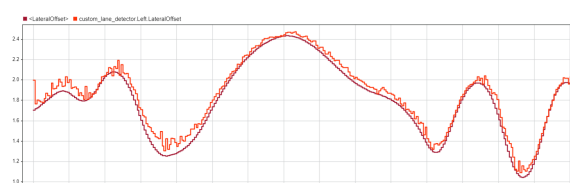


Figure 11: Left Lateral Offset: ground truth (purple) and custom estimation (orange).

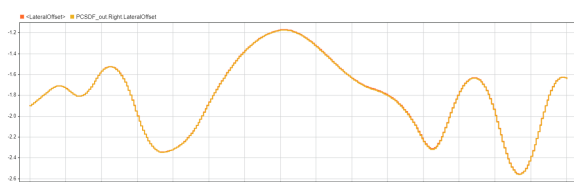


Figure 12: Right Lateral Offset: ground truth (orange) and original estimation (yellow).

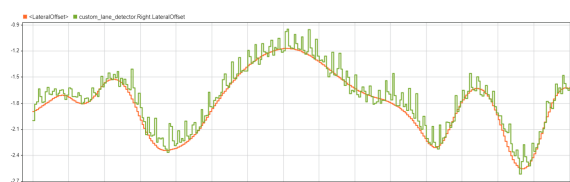


Figure 13: Right Lateral Offset: ground truth (orange) and custom estimation (green).

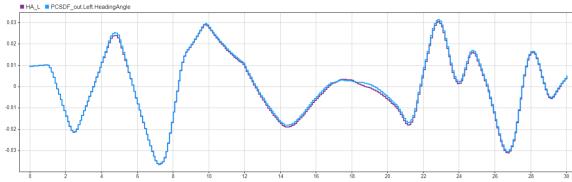


Figure 14: Left Heading Angle: ground truth (purple) and original estimation (blue).

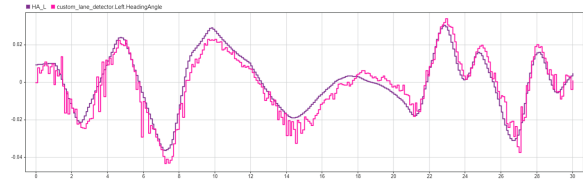


Figure 15: Left Heading Angle: ground truth (purple) and custom estimation (pink).

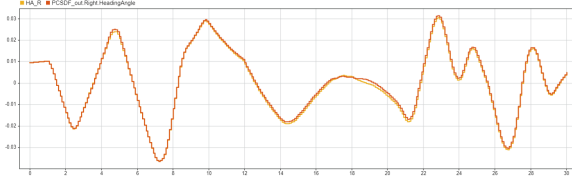


Figure 16: Right Heading Angle: ground truth (yellow) and original estimation (red).

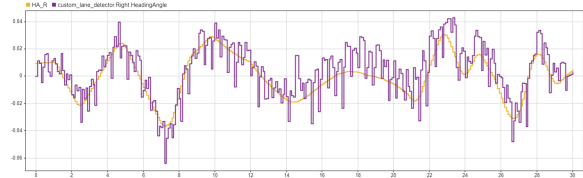


Figure 17: Right Heading Angle: ground truth (yellow) and custom estimation (purple).

As we can see, the original estimation is almost perfect and the imperfections are caused by the noise that is added on purpose. The estimation from our custom implementation isn't as good but it manages to follow the shape of the lane correctly, with a small offset, both on the lateral offset and on the heading angle. We decided to use the original estimation for all the other tests as it's the most stable.

Controller

The controller is responsible for calculating the LTA's desired steering with the **lateral offset** (from the lane's center line) and **heading angle** (or yaw angle) as inputs.

We decided to implement a PD Controller, as it is a simple but effective solution for our task. This controller has 2 gains:

1. **Proportional Gain (K_P):** This is the gain related to the proportional error. In our context, the lateral offset. This term ensures the vehicle is pulled towards the reference.
2. **Derivative Gain (K_D):** This is the gain related to the rate of change of the error. In our context, the heading angle, which acts as the derivative of the lateral offset¹. This term introduces damping, minimizing the overshoot.

The resulting control law for the steering angle $\delta(t)$ can be expressed as:

$$\delta(t) = -K_P \cdot e(t) - K_D \cdot \dot{e}(t) \quad (5)$$

These gains have to be tuned, in order to provide a good balance of responsiveness and stability. We tuned the gains empirically, aiming for a response that ensures the

¹The derivative is also dependent on the longitudinal velocity, but we can neglect it because the controller is designed for highway use, so a limited velocity range

vehicle does not cross the lane boundary, while not oscillating aggressively, which would compromise passenger comfort and stability of the car.

$$K_P = 0.2 \quad K_D = 2 \quad (6)$$

Why not use a PID Controller? The integral component is used to eliminate steady-state error. In our model there is no steady-state error and the integral component would accumulate error causing overshoot.

LTA Activation

Our LTA system is designed as a safety system and not for autonomous driving. This means it only actuates when detecting a driver error. It was also designed as to transition smoothly from driver to the LTA controller's target steering. Furthermore, we opted to allow the driver to resist the LTA actuation and perform a lane change without using a blinker, because it might be needed for emergency avoiding actions. This part of the system is implemented on the subsystem *Apply Assist* 18.

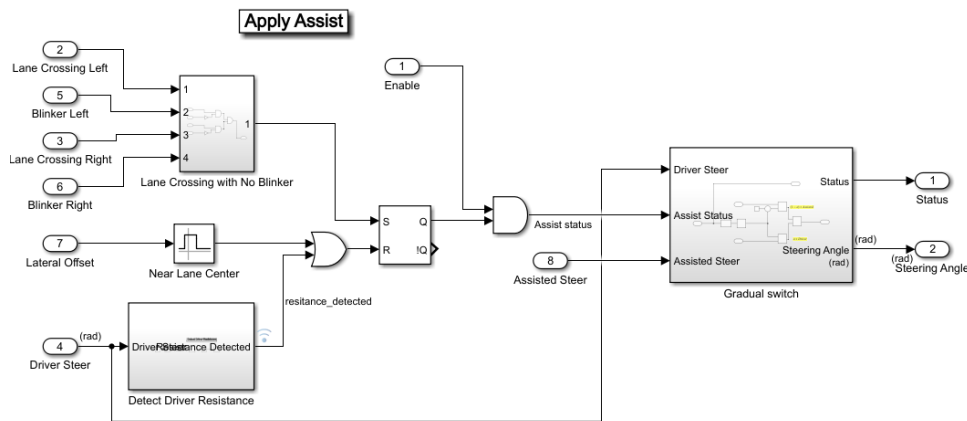


Figure 18: Apply Assist

The activation follows a S-R Flip-Flop logic, and the state transitions are smoothened by the Gradual Switch block.

Set

The LTA is activated when lane crossing is detected. Lane crossing is defined by the car exceeding the safety lateral distance to either lane. However, if the blinker is activated for the crossing direction, it overrides the detection and the LTA is not activated. This allows the driver to change lanes with no disturbance.

Reset

The driver regains control of the steering when either of these situations is detected:

1. The car is near the lane center again, after the LTA has corrected the steering.
2. The driver is resisting the LTA actuation

Driver Resistance

In a real car, resistance of the driver against the LTA would be detected by measuring his steering torque. The best method we found to replicate this with a gamepad was to detect if the steering input is at the maximum (either left or right) for 5 consecutive samples.

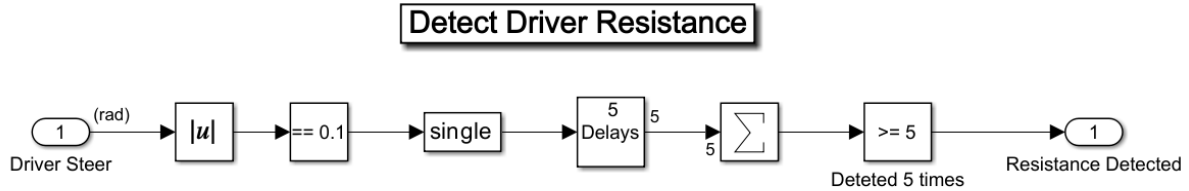


Figure 19: Driver Resistance Detection

Gradual Switch

Similarly to the driver resistance detection, we had to find a method to simulate the gradual steering change produced by the LTA's steering torque. We decided to implement a rate limiter, which results in a linear transition from the driver's steering to the LTA's target steering and the same inversely. [20](#)

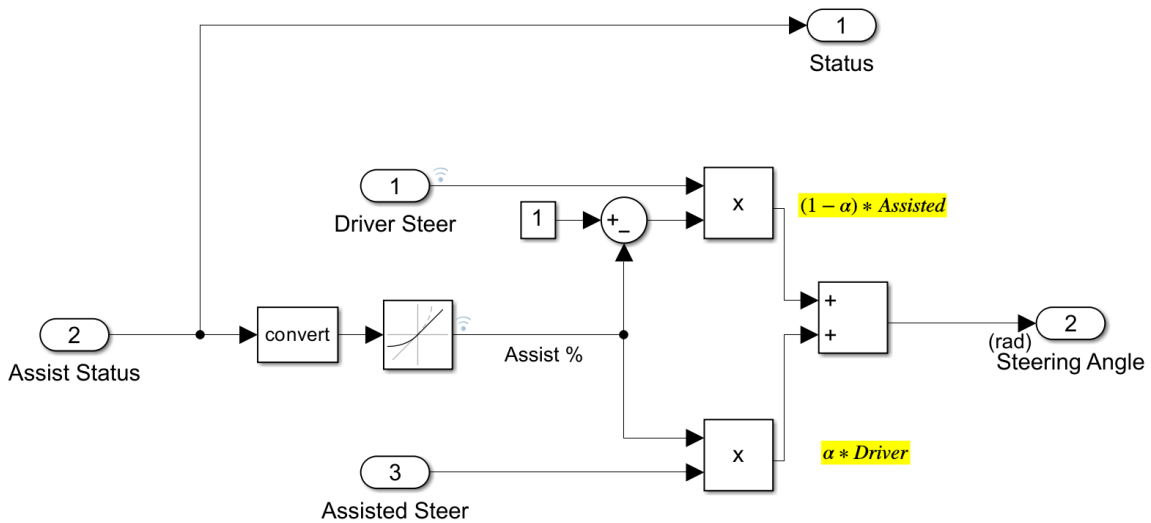


Figure 20: Rate Limiter

Effectiveness of the proposed LTA

To prove that our LTA system is indeed effective, it must be classified as stable. As we expressed previously, our PD controller is represented by:

$$\delta = -K_p \cdot e_1 - K_d \cdot e_2 \quad (7)$$

the value for δ will be negative, since its feedback is supposed to be negative. To help out with this, we use the *Lyapunov* Direct Method. This method defines a scalar function

$V(e)$ of the systems state. In order for the system to be designated as stable, the function must follow these properties:

- $V(e) > 0 \quad \forall e \neq 0$
- $V(e) = 0 \quad e = 0$
- $\dot{V}(e) < 0 \quad \forall e \neq 0$
- $V(e) \rightarrow \infty \quad \|x\| \rightarrow \infty$

The vehicle dynamics will be:

$$\dot{e}_1 = v \sin(e_2) + v_y \quad (8)$$

$$\dot{e}_2 = \frac{v}{L} \tan(\delta) \quad (9)$$

The derivative of the lateral deviation e_1 will be equal to the lateral velocity of the vehicle plus the forward velocity along the lane direction. By assuming a bicycle model, this last term can be neglected. The derivative of the heading angle e_2 will be the same as the variation of steering over time. Both e_2 and δ will be small values, so it is possible to simplify both expressions:

$$\dot{e}_1 = v \cdot e_2 \quad (10)$$

$$\dot{e}_2 = \frac{v}{L} \cdot \delta \quad (11)$$

The Lyapunov equation that we decide to use is the following:

$$V(e_1, e_2) = \frac{1}{2} a e_1^2 + \frac{1}{2} b e_2^2 + c e_1 e_2 \quad (12)$$

To prove that its positive definite, we analyze it as a quadratic form and Write it in matrix form

$$V = \begin{bmatrix} e_1 & e_2 \end{bmatrix} \underbrace{\begin{bmatrix} a/2 & c/2 \\ c/2 & b/2 \end{bmatrix}}_P \begin{bmatrix} e_1 \\ e_2 \end{bmatrix}$$

The function V is positive definite if and only if the matrix P is positive definite. This holds when:

1. The leading principal minor is positive:

$$\frac{a}{2} > 0 \quad \implies \quad a > 0$$

2. The determinant of P is positive:

$$\det(P) = \frac{ab - c^2}{4} > 0 \quad \implies \quad ab > c^2$$

Therefore, the Lyapunov function is positive definite if

$$a > 0, \quad b > 0, \quad ab > c^2$$

These conditions ensure that $V(e_1, e_2) > 0$ for all $(e_1, e_2) \neq (0, 0)$, regardless of the signs of e_1 and e_2 .

The Lyapunov function is radially unbounded since the quadratic form is positive definite (i.e., $c^2 < ab$), then the smallest eigenvalue $\lambda_{\min}(P)$ of the matrix P satisfies

$$V(e) \geq \lambda_{\min}(P) \|e\|^2$$

Thus,

$$\|e\| \rightarrow \infty \Rightarrow V(e) \rightarrow \infty$$

This ensures that the Lyapunov function is coercive and suitable for global stability analysis.

Differentiating the Lyapunov function yields

$$\dot{V} = ae_1\dot{e}_1 + be_2\dot{e}_2 + c(\dot{e}_1e_2 + e_1\dot{e}_2)$$

Which becomes:

$$\dot{V} = ae_1(v\dot{e}_2) + be_2\left(\frac{v}{L}(-K_p e_1 - K_d e_2)\right) + c\left(v\dot{e}_2e_2 + e_1\frac{v}{L}(-K_p e_1 - K_d e_2)\right)$$

After grouping the terms, we obtain a quadratic form:

$$\dot{V} = -\alpha e_1^2 - \beta e_2^2 - \gamma e_1 e_2,$$

where $\alpha, \beta, \gamma > 0$ depend on a, b, c, K_p, K_d, v, L .

The derivative is negative definite if and only if the symmetric matrix

$$Q = \begin{bmatrix} \alpha & \frac{\gamma}{2} \\ \frac{\gamma}{2} & \beta \end{bmatrix}$$

is positive definite.

Thus, we require:

$$\alpha > 0, \quad \beta > 0, \quad \gamma^2 < 4\alpha\beta$$

Under these conditions:

$$\dot{V}(e_1, e_2) < 0 \quad \forall (e_1, e_2) \neq (0, 0),$$

which guarantees asymptotic stability of the closed-loop Lateral Tracking system.

Role of each author

Beatriz Coelho: Car Kinematic Model

Daniel Burke: Effectiveness of the LTA

Diogo Sousa: Controller, LTA Activation, Gamepad integration

Miguel Fialho: Sensors and Perception, 3D Simulation

Bibliography

- [1] - [Lane Keeping Assist with Lane Detection Example](#)
- [2] - [Vision Detection Generator Documentation](#)
- [3] - [Highway Lane Following Example](#)
- [4] - [Lane-Detection-using-MATLAB](#)
- [5] - [Udacity-Advance-Lane-detection-of-the-road](#)